

Change of Basis and Coordinates of vectors

Meenu S

January 1, 2026

1 Introduction

This document provides a detailed explanation of a Python program that computes the change of basis between two vector spaces, calculates the transition matrix, and visualizes the result. The program performs the following tasks:

- Accepts the vector space dimension (2D or 3D).
- Prompts the user for two basis vectors.
- Computes the transition matrix from the initial basis to the new basis.
- Visualizes the vector in both bases, and generates plots.

Change of Basis and Coordinate Matrices

Let V be a finite-dimensional vector space over a field $\mathbb{F}$ and let $B = \{b_1, b_2, \dots, b_n\}$ be an ordered basis of V .

Every vector $v \in V$ can be written uniquely as

$$v = c_1 b_1 + c_2 b_2 + \dots + c_n b_n,$$

where $c_i \in \mathbb{F}$. The coordinate vector of v with respect to the basis B is

$$[v]_B = \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix}.$$

Transition Matrix Between Bases

Let $B = \{b_1, b_2, \dots, b_n\}$ and $C = \{c_1, c_2, \dots, c_n\}$ be two ordered bases of a vector space V .

Define the basis matrices

$$[B] = (b_1 \ b_2 \ \dots \ b_n), \quad [C] = (c_1 \ c_2 \ \dots \ c_n).$$

(The basis elements are column vectors)

For any vector $v \in V$, we have

$$v = [B] [v]_B \quad \text{and} \quad v = [C] [v]_C.$$

Equating the two expressions,

$$[B] [v]_B = [C] [v]_C.$$

Multiplying both sides by $[C]^{-1}$, we obtain

$$[C]^{-1}[B] [v]_B = [v]_C.$$

Hence,

$$[v]_C = P_{C \leftarrow B} [v]_B,$$

where the transition matrix from basis B to basis C is

$$P_{C \leftarrow B} = [C]^{-1}[B].$$

Inverse Transition Matrix

The transition matrix from C to B is the inverse of $P_{C \leftarrow B}$:

$$P_{B \leftarrow C} = (P_{C \leftarrow B})^{-1},$$

and hence

$$[v]_B = P_{B \leftarrow C} [v]_C.$$

*Let the new basis be

$$B = \{b_1, b_2\} = \{(\cos x, \sin x), (-\sin x, \cos x)\}$$

and let the vector in standard coordinates be

$$v = (1, 1).$$

$$P_B = [b_1 \ b_2] = \begin{bmatrix} \cos x & -\sin x \\ \sin x & \cos x \end{bmatrix}$$

By definition, the coordinate vector $[v]_B$ satisfies

$$v = P_B [v]_B \quad \implies \quad [v]_B = P_B^{-1} v$$

Since P_B is orthogonal (rotation matrix),

$$P_B^{-1} = P_B^\top = \begin{bmatrix} \cos x & \sin x \\ -\sin x & \cos x \end{bmatrix}$$

$$[v]_B = P_B^\top v = \begin{bmatrix} \cos x & \sin x \\ -\sin x & \cos x \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos x + \sin x \\ -\sin x + \cos x \end{bmatrix} = \begin{bmatrix} \cos x + \sin x \\ \cos x - \sin x \end{bmatrix}$$

$$\cos 45^\circ = \sin 45^\circ = \frac{1}{\sqrt{2}}$$

$$[v]_B = \begin{bmatrix} \frac{1}{\sqrt{2}} + \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} - \frac{1}{\sqrt{2}} \end{bmatrix} = \begin{bmatrix} \sqrt{2} \\ 0 \end{bmatrix}$$

When (1,1) is rotated anti-clockwise by 45 degrees, it will intersect the x-axis at this point (length of the vector (1,1) is $\sqrt{2}$ - Rotation as change of basis)

2 Python Code Listing

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import os
4
5 def get_basis_from_user(dim, basis_name):
6     """Prompts the user to enter basis vectors."""
7     print(f"--- Enter the {dim} {basis_name} vectors ---")
8     basis_vectors = []
9     for i in range(dim):
10         while True:
11             try:
12                 b_str = input(f"Basis vector {i+1}: ").split()
13                 if len(b_str) != dim:
14                     print(f"Error: Please enter {dim} components.")
15                     continue
16                 basis_vectors.append([float(x) for x in b_str])
17                 break
18             except ValueError:
19                 print("Invalid input. Please enter numbers only.")
20     # Create matrix with basis vectors as columns
21     matrix = np.array(basis_vectors).T
22     # Check if the basis is valid (i.e., vectors are linearly
23     # independent)
24     if abs(np.linalg.det(matrix)) < 1e-9: # Use a tolerance for
25         # floating point numbers
26         print(f"\nError: The vectors provided for '{basis_name}' are
27             not linearly independent and do not form a valid basis."
28             )
29     return None
30
31 return matrix
32
33 def general_change_of_basis():

```

```

29 """
30 Performs a change of basis for a vector from one arbitrary basis
    to another.
31 """
32 try:
33     # 1. Specify the dimension
34     dim_str = input("Enter the dimension of the vector space (2
        or 3): ")
35     dim = int(dim_str)
36     if dim not in [2, 3]:
37         print("Invalid input. This script can only plot for
            dimensions 2 and 3.")
38         return
39
40     # 2. Get the initial basis (B)
41     B = get_basis_from_user(dim, "initial basis (B)")
42     if B is None: return
43
44     # 3. Get the vector's coordinates in the initial basis
45     print("\nEnter the vector's coordinates relative to the
        initial basis B, separated by spaces:")
46     v_coords_B_str = input("Vector coordinates: ").split()
47     v_coords_B = np.array([float(x) for x in v_coords_B_str]).
        reshape(-1, 1) # Column vector
48
49     # 4. Get the new basis (B')
50     B_prime = get_basis_from_user(dim, "new basis (B')")
51     if B_prime is None: return
52
53     # 5. Define the transition matrix  $C = (B')^{-1} * B$ 
54     B_prime_inv = np.linalg.inv(B_prime)
55     C = B_prime_inv @ B
56
57     # 6. Calculate the new coordinates:  $v\_coords\_B' = C * v\_coords\_B$ 
58     v_coords_B_prime = C @ v_coords_B
59
60     # 7. Print the results
61     np.set_printoptions(precision=3, suppress=True)
62     print("\n" + "="*40)
63     print("                        RESULTS")
64     print("="*40)
65     print(f"Transition Matrix from B to B' (C):\n{C}\n")
66     print(f"Original coordinates in basis B: {v_coords_B.flatten()
        ()}")
67     print(f"New coordinates in basis B': {v_coords_B_prime.
        flatten()}")

```

```

68     print("="*40)
69
70     # 8. Generate plots for 2D or 3D cases
71     # The actual geometric vector in standard coordinates is B @
       v_coords_B
72     v_geometric = (B @ v_coords_B).flatten()
73
74     print(f"\nGenerating {dim}D plots...")
75     fig = plt.figure(figsize=(16, 8))
76     save_path = 'general_basis_change.png'
77
78     if dim == 2:
79         ax1 = fig.add_subplot(1, 2, 1)
80         ax2 = fig.add_subplot(1, 2, 2)
81
82         # --- Plot 1: Vector in Initial Basis B ---
83         ax1.set_title("Vector in Initial Basis (B)")
84         ax1.quiver(0, 0, B[0, 0], B[1, 0], angles='xy',
85                   scale_units='xy', scale=1, color='g', label=f'b1 = {B
86                     [:,0]}')
87         ax1.quiver(0, 0, B[0, 1], B[1, 1], angles='xy',
88                   scale_units='xy', scale=1, color='b', label=f'b2 = {B
89                     [:,1]}')
90         ax1.quiver(0, 0, v_geometric[0], v_geometric[1], angles=
91                   'xy', scale_units='xy', scale=1, color='r', label=f"
92                   Coords = {v_coords_B.flatten()}")
93
94         # --- Plot 2: Vector in New Basis B' ---
95         ax2.set_title("Vector in New Basis (B')")
96         ax2.quiver(0, 0, B_prime[0, 0], B_prime[1, 0], angles='
97                   xy', scale_units='xy', scale=1, color='g', label=f"b
98                   '1 = {B_prime[:,0]}")
99         ax2.quiver(0, 0, B_prime[0, 1], B_prime[1, 1], angles='
100                  xy', scale_units='xy', scale=1, color='b', label=f"b
101                  '2 = {B_prime[:,1]}")
102         ax2.quiver(0, 0, v_geometric[0], v_geometric[1], angles=
103                  'xy', scale_units='xy', scale=1, color='r', label=f"
104                  Coords = {v_coords_B_prime.flatten()}")
105
106         all_vectors = np.hstack([B, B_prime, v_geometric.reshape
107                                  (-1,1)])
108         max_val = np.max(np.abs(all_vectors)) + 1
109         for ax in [ax1, ax2]:
110             ax.set_aspect('equal', adjustable='box')
111             ax.set_xlim(-max_val, max_val); ax.set_ylim(-max_val
112                   , max_val)
113             ax.grid(); ax.legend()

```

```

100         ax.axhline(0, color='k', lw=0.5); ax.axvline(0,
101                 color='k', lw=0.5)
102
103     elif dim == 3:
104         ax1 = fig.add_subplot(1, 2, 1, projection='3d')
105         ax2 = fig.add_subplot(1, 2, 2, projection='3d')
106
107         # --- Plot 1: Vector in Initial Basis B ---
108         ax1.set_title("Vector in Initial Basis (B)")
109         ax1.quiver(0, 0, 0, B[0,0], B[1,0], B[2,0], color='g',
110                 label=f'b1')
111         ax1.quiver(0, 0, 0, B[0,1], B[1,1], B[2,1], color='b',
112                 label=f'b2')
113         ax1.quiver(0, 0, 0, B[0,2], B[1,2], B[2,2], color='y',
114                 label=f'b3')
115         ax1.quiver(0, 0, 0, v_geometric[0], v_geometric[1],
116                 v_geometric[2], color='r', label=f'Coords = {
117                 v_coords_B.flatten()}')
118
119         # --- Plot 2: Vector in New Basis B' ---
120         ax2.set_title("Vector in New Basis (B')")
121         ax2.quiver(0, 0, 0, B_prime[0,0], B_prime[1,0], B_prime
122                 [2,0], color='g', label=f"b'1")
123         ax2.quiver(0, 0, 0, B_prime[0,1], B_prime[1,1], B_prime
124                 [2,1], color='b', label=f"b'2")
125         ax2.quiver(0, 0, 0, B_prime[0,2], B_prime[1,2], B_prime
126                 [2,2], color='y', label=f"b'3")
127         ax2.quiver(0, 0, 0, v_geometric[0], v_geometric[1],
128                 v_geometric[2], color='r', label=f"Coords = {
129                 v_coords_B_prime.flatten()}")
130
131         all_vectors = np.hstack([B, B_prime, v_geometric.reshape
132                 (-1,1)])
133         max_val = np.max(np.abs(all_vectors)) + 1
134         for ax in [ax1, ax2]:
135             ax.set_xlim(-max_val, max_val); ax.set_ylim(-max_val
136                     , max_val); ax.set_zlim(-max_val, max_val)
137             ax.set_xlabel('X'); ax.set_ylabel('Y'); ax.
138                 set_zlabel('Z')
139             ax.legend()
140
141     fig.tight_layout(pad=3.0)
142     try:
143         plt.savefig(save_path)
144         if os.path.exists(save_path):
145             print(f"Success! Plot saved as {save_path}.")
146         else:

```

```

133         print(f"Error: Plot file not created.")
134     except Exception as e:
135         print(f"--- FAILED TO SAVE PLOT: {e} ---")
136
137     except ValueError:
138         print("Invalid input. Please ensure you enter numbers only.")
139
140     except np.linalg.LinAlgError as e:
141         print(f"A matrix error occurred: {e}. One of the bases might
142             be invalid.")
143     except Exception as e:
144         import traceback
145         print(f"--- AN UNEXPECTED ERROR OCCURRED: {e} ---")
146         print(f"TRACEBACK: {traceback.format_exc()}")
147
148 if __name__ == '__main__':
149     # Activate the virtual environment first by running:
150     # source .venv/bin/activate
151     general_change_of_basis()

```

Listing 1: Python Program for Change of Basis and Transition Matrices

OUTPUT

Enter the dimension of the vector space (2 or 3): 3

— Enter the 3 initial basis (B) vectors —

Basis vector 1: 1 1 1

Basis vector 2: 1 -1 1

Basis vector 3: 0 0 1

Enter the vector's coordinates relative to the initial basis B, separated by spaces:

Vector coordinates: 6 -1 -1

— Enter the 3 new basis (B') vectors —

Basis vector 1: 2 2 0

Basis vector 2: 0 1 1

Basis vector 3: 1 0 1

Transition Matrix from B to B' (C):

$$\begin{bmatrix} 0.25 & -0.25 & -0.25 \\ 0.5 & -0.5 & 0.5 \\ 0.5 & 1.5 & 0.5 \end{bmatrix}$$

Original coordinates in basis B: [6. -1. -1.]

New coordinates in basis B': [2. 3. 1.]

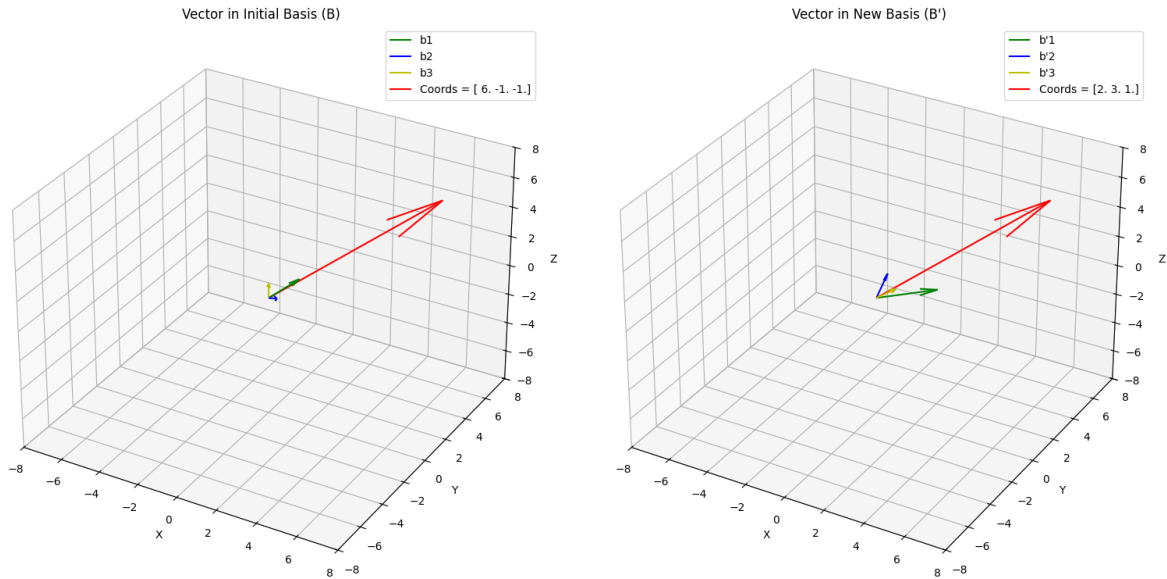


Figure 1: Representation of the vector in 2 different bases

3 Explanation of the Code (Notes)

Import Statements

- `import numpy as np`: Imports the NumPy package for numerical operations, especially matrix and vector calculations.
- `import matplotlib.pyplot as plt`: Imports Matplotlib for generating plots of vectors in the different bases.
- `import os`: Used to check if the plot file is successfully saved.

The Function `get_basis_from_user`

- This function prompts the user to enter basis vectors.
- The matrix is created using these vectors as columns. It checks if the basis vectors are linearly independent by checking if the determinant of the matrix is non-zero.
- If the determinant is zero, the vectors are linearly dependent, and the function returns `None`.

The Function `general_change_of_basis`

- This function handles the entire process of changing the basis, including obtaining the initial and new bases, calculating the transition matrix, and visualizing the result.

Step-by-step Breakdown

- `dim = int(input("Enter the dimension of the vector space (2 or 3): "))`: Takes the dimension of the vector space as input (either 2D or 3D).
- `B = get_basis_from_user(dim, "initial basis (B)")`: Calls the `get_basis_from_user` function to obtain the initial basis vectors.
- `v_coords_B = np.array([float(x) for x in v_coords_B_str]).reshape(-1,1)`: Converts the input vector coordinates into a NumPy array.
- `C = np.linalg.inv(B_prime) @ B`: Calculates the transition matrix C , where $C = (B')^{-1} \times B$.
- `v_coords_B_prime = C @ v_coords_B`: Calculates the vector's new coordinates in the new basis by multiplying the transition matrix with the original coordinates.
- The program then prints the transition matrix and the original and new coordinates.
- Finally, it generates 2D or 3D plots of the vectors in the initial and new bases using Matplotlib.